Scenario: Real-time FinTech transaction monitoring and fraud reporting using a Kappa-style pipeline.

Name: Gimnadhi P.M.T.

Reg No: EG/2020/3942

1. Introduction & Scenario Overview

This mini project implements an end-to-end streaming data pipeline that simulates a real FinTech environment where transaction events arrive continuously and must be processed for immediate detection of suspicious activity while also supporting periodic analytics and reporting. The pipeline follows a Kafka architecture approach: streaming is the primary processing path, and historical reporting is generated from stored streaming outputs rather than maintaining a separate batch-processing codebase.

A Python producer generates mock transaction events containing fields such as transaction_id, user_id, timestamp, merchant_category, amount, country, and city. These events are sent to Kafka for reliable ingestion and decoupling between producers and consumers. Spark Structured Streaming consumes the Kafka topic, parses the data into a structured schema, derives event_time, and applies fraud detection rules. The results are persisted into PostgreSQL tables (for queryable reporting) and into Parquet storage (for efficient historical analysis). Finally, an Airflow DAG is used to orchestrate the reporting pipeline every 6 hours, producing CSV outputs such as reconciliation summaries and fraud analytics (e.g., fraud attempts by merchant category).

2. Justification of Tool Selection (Tech Stack)

2.1 Apache Kafka (Ingestion Layer)

Kafka was selected as the ingestion system because it is designed for high-throughput, durable event streaming. It provides key features required in real-world data engineering:

- **Decoupling:** Producers (data generators) and consumers (Spark streaming jobs) are loosely coupled. Producers can keep sending events even if consumers temporarily fail or restart.
- **Durability and replay:** Kafka retains messages for a configured duration, enabling reprocessing and recovery. This is important for debugging, auditing, and handling downstream failures.
- **Scalability through partitions:** Kafka topics can be partitioned to scale both throughput and consumer parallelism. Spark can read multiple partitions in parallel via its Kafka source connector.
- **Real-time behavior:** Kafka supports near-real-time ingestion, which is essential for fraud detection scenarios.

Kafka therefore fits the requirement to simulate a real-time event-driven environment where the pipeline must process streaming data continuously.

2.2 Apache Spark Structured Streaming (Stream Processing Layer)

Spark Structured Streaming was chosen for stream processing because it provides a mature and developer-friendly framework for handling real-time data with strong integration into the broader Spark ecosystem:

- **Structured API:** Instead of low-level event-by-event handling, Spark provides high-level transformations on streaming DataFrames, making the logic clearer and easier to maintain.
- **Event-time support:** Spark supports event-time processing, windowing, and watermarks, which are crucial in fraud detection and other time-sensitive analytics.
- **Fault tolerance with checkpointing:** Structured Streaming supports checkpointing to allow recovery after failures and ensure consistent processing.
- **Integration with sinks:** Spark can write to JDBC (PostgreSQL) and file formats like Parquet efficiently, enabling hybrid storage choices.

This made Spark a suitable choice for implementing fraud rules, enrichment logic, and writing to storage in a reliable and scalable way.

2.3 PostgreSQL and Parquet (Storage/Sink Layer)

The project uses **PostgreSQL** and **Parquet** to satisfy different storage needs:

PostgreSQL:

- Provides a relational schema for structured data such as raw_transactions and fraud_alerts.
- Supports SQL queries required by the reporting stage.
- Makes it easy to integrate with Airflow tasks for report generation.

Parquet:

- Provides an efficient, columnar format for historical/curated storage.
- Enables scalable analytical reads compared to row-based storage.
- Acts as a warehouse-like storage layer for validated transactions.

Using both allows fast operational reporting (Postgres) while also supporting efficient long-term analytical storage (Parquet). This combination resembles real production systems where OLTP/operational stores coexist with analytical warehouses.

2.4 Apache Airflow (Orchestration Layer)

Airflow was selected for orchestration and scheduled analytics because it is built specifically for workflow scheduling and monitoring:

- **Scheduling:** The DAG can run automatically every 6 hours (or be triggered manually).
- **Observability:** Airflow provides a UI to view task logs, retries, execution history, and failures.

- **Dependency management:** Tasks like validation, aggregation, and report export can be ordered and separated cleanly.
- **Repeatable reporting:** Report generation is consistent and reproducible because it runs as code with a defined schedule and environment.

In this project, Airflow orchestrates periodic validation and report generation, producing CSV files used as the final deliverable.

3. Event Time vs Processing Time Handling

3.1 Definitions

- **Event Time:** The time when the transaction actually occurred (embedded in the record as timestamp).
- **Processing Time:** The time when Spark receives and processes the event.

These two differ when there are delays in producing events, Kafka lag, network issues, or consumer downtime.

3.2 Implementation Approach

This pipeline explicitly prioritizes **event time** to ensure correct fraud detection logic. The producer includes a timestamp value in each record. In Spark Structured Streaming:

1. The timestamp field is parsed and converted into a proper timestamp column called event_time:
    - Example: event_time = to_timestamp(col("timestamp"), "...")
2. The system applies a **watermark** on event_time (e.g., 10 minutes):
    - withWatermark("event_time", "10 minutes")

3.3 Why Watermarking Matters

Fraud detection rules such as “impossible travel within 10 minutes” depend on the actual event time. If processing time were used, a late event could be incorrectly classified or ignored simply because it arrived late, even though logically it belongs within the fraud window.

Watermarking allows Spark to:

- **Accept late events** up to a certain delay threshold (10 minutes in this case).
- **Bound state growth** by eventually discarding old state beyond the watermark to keep memory usage stable.

3.4 Trade-Off

A larger watermark improves correctness for late data but increases memory/state requirements. A smaller watermark reduces memory usage but may drop legitimately late events. The project uses a moderate watermark window to balance realism and stability for a simulated environment.

4. Ethics, Privacy, and Data Governance (Required Section)

4.1 Privacy Implications in the Scenario

Although the data is simulated, the scenario models a real FinTech pipeline, which in practice deals with highly sensitive information. Potential privacy risks include:

- **User profiling:** merchant_category can reveal personal habits, lifestyle patterns, or sensitive interests (e.g., medical purchases, religious donations, gambling).
- **Location tracking:** country and city data can be used to infer travel behavior and possibly identify individuals through mobility patterns.
- **Financial sensitivity:** Even without exact account details, transaction amounts can reveal economic status, spending behavior, and vulnerability.
- **Linkability:** A stable user_id across transactions allows long-term tracking and correlation, which can reduce anonymity.
- **False positives and harm:** Fraud detection flags can lead to account restrictions or investigations. If inaccurate, these can unfairly impact users.

4.2 Data Governance Measures (What should be applied)

To handle these risks responsibly, the following governance practices should be applied in a real deployment:

1. **Data minimization:** Only collect the fields required for fraud detection and reporting. If city-level detail is not needed, use country-only or region-level data.
2. **Pseudonymization:** Replace direct user identifiers with hashed or tokenized identifiers to reduce re-identification risk.
3. **Access control:** Restrict who can access raw data tables. Analysts should typically access aggregated results rather than raw transaction logs.
4. **Retention policies:** Store raw transaction-level data only for a defined period required for fraud investigation, then archive or delete it.
5. **Encryption:**
    - Encrypt data in transit (Kafka/Spark/Postgres TLS where possible).
    - Encrypt data at rest (database storage encryption and secure file storage).

6. **Audit logging:** Track who accessed sensitive datasets and when, especially for fraud investigations.
7. **Transparency and accountability:** In real systems, users should be informed about fraud monitoring practices and how their data is used.
8. **Fairness checks:** Fraud rules should be evaluated to prevent disproportionate targeting of certain locations/categories, and the system should be calibrated to reduce bias and false positives.

4.3 Ethical Summary

Fraud detection can protect users and reduce financial crime, but it must be implemented in a way that respects privacy, avoids excessive surveillance, and ensures governance policies are applied. Ethical data engineering requires balancing security needs with user rights, transparency, and responsible data handling.

5. Conclusion

This project demonstrates an end-to-end Kappa-style streaming pipeline using Kafka, Spark Structured Streaming, PostgreSQL/Parquet, and Airflow. Kafka enables reliable real-time ingestion and replay; Spark performs structured, event-time-aware fraud detection with watermarking; PostgreSQL supports reporting queries while Parquet supports efficient historical storage; and Airflow orchestrates periodic analytics and CSV report generation. The pipeline satisfies both real-time insight requirements (fraud detection) and historical analysis/reporting needs. Finally, the project acknowledges important ethical and privacy concerns and outlines governance measures needed to apply such a system responsibly in real-world FinTech environments.